現場最前線から教える データサイエンス 1

ITベンダーにおけるデータサイエンティスト
2025年度　〇〇大学

日本IBMシステムズ・エンジニアリング株式会社
DXセンター データサイエンスラボ
川口 英俊

自己紹介

- 講師名: 川口 英俊 (かわぐち ひでとし)
- 所属:
  - 日本IBMシステムズエンジニアリング(株) DXセンター データサイエンスラボ（Experienced Data Scientist)
  - 情報処理学会　論文誌ジャーナル編集委員会 知能グループ（編集委員, メタ査読者)
- 学位: 博士（情報科学)
- 経歴
  - 2010年-2014年 広島工業大学 情報学部 知的情報システム学科 (学士)
  - 2014年-2016年 東京工業大学 総合理工学研究科 知能システム科学専攻博士前期課程 (修士)
  - 2016年-2023年 日本電信電話株式会社(現: NTT株式会社)
    - 在籍中2019年-2023年: 北陸先端科学技術大学院大学 先端科学技術研究科 先端情報科学プログラム 博士後期課程(博士)
  - 2023年 7月より現職
- 専門: 機械学習, データ分析, 自然言語処理, ソフトウェア開発, AWS
- 代表的な仕事:
  - IPSのシグネチャ設定を支援する機械学習の応用研究
  - 生成AIと機械学習を利用したVoC (Voice of Customers) 分析パイプライン処理の開発

github.com/HidetoshiKawaguchi | 𝕏 @Hidetoshi_RM | in linkedin.com/in/hidetoshi-kawaguchi-380a93281

本資料の注意事項

- 本資料は教育目的で公開しています
- 本資料はとある大学で学生向けに講義した内容を流用し作成されています
  - 大学からの希望があり、大学名は伏せています
  - 講義名は架空のものに置き換えています
- 資料内容はすべて川口個人の見解によるものであり、所属組織の公式見解と異なることがあることにご注意ください

川口担当分の講義の概要・目的

人工知能・機械学習・確率統計等の専門技能を、プロのデータサイエンティストとして活かし成果に結びつけるための基礎的な知識・考え方・実践力を養います。

- 学ぶこと
  - ITベンダーにおけるデータサイエンティストとは何か
  - ドキュメント・コミュニケーション
  - 分析・実験・検証の実践的管理方法
  - データサイエンティストの開発技術
- 学ばないこと
  - 最新の人工知能・機械学習・確率統計のような専門知識
  - 最新のクラウド技術を使った開発手法

講師としての想い

2, 3年後には使えなくなっているような知識ではなく、10年、20年先も役立つ根源的なスキルを習得してもらいたいです。

身につけるためにはすごく訓練が必要ですが、これから学生として研究をしていったり、仕事をする中でちょっとでも意識してもらえると幸いです。

4回分の講義の目次

- **ITベンダーにおけるデータサイエンティスト**
  ITベンダーにおけるデータサイエンティストの役割と、以降の講義の流れ・前提知識・ツールやライブラリの使い方について学ぶ。

- **ドキュメント・コミュニケーション**
  データサイエンティストが知っておくべきドキュメントの書き方やプレゼンテーション資料の作成方法について学ぶ

- **分析・実験・検証の実践的管理方法**
  データサイエンティストの実施する仮説検証のプラクティカルな管理方法について学ぶ。

- **データサイエンティストの開発技術**
  データサイエンティストが開発現場で最低限知っておきたい実践的な技術について学ぶ。

今回の講義の目次

- [前半] イントロダクション
  - 本講義で学ぶデータサイエンティストとは何か
  - ITベンダーのデータサイエンティストの仕事
  - ITベンダーのデータサイエンティストと周囲の人々
- [後半] 2回目以降の講義のためのセットアップ・前提知識
  - Pythonのバージョン管理
  - 仮想環境の管理

今回の講義の目次

- **[前半] イントロダクション**
  - **本講義で学ぶデータサイエンティストとは何か**
  - **ITベンダーのデータサイエンティストの仕事**
  - **ITベンダーのデータサイエンティストと周囲の人々**
- [後半] 2回目以降の講義のためのセットアップ・前提知識
  - Pythonのバージョン管理
  - 仮想環境の管理

最初に、「データサイエンティストとは何か？」という問いに調べたり考えたりしてみましょう

自身の職業について哲学することは、非常に大事なことです

また、**本講義で学ぶ対象となるデータサイエンティスト像を解説していきます。**

データサイエンティストとは何か –色々な組織の定義-

データサイエンティストの一般化は難しく、色々な組織で色々な説明がされています。なかなか統一見解を作るのは難しい

組織	説明文	ソース
情報処理推進機構 (IPA)	DXの推進において、データを活用した業務変革や新規ビジネスの実現に向けて、データを収集・解析する仕組みの設計・実装・運用を担う人材	https://www.ipa.go.jp/jinzai/skill-standard/dss/datascientist/index.html
一般社団法人データサイエンティスト協会	データサイエンス力、データエンジニアリング力をベースにデータから価値を創出し、ビジネス課題に答えを出すプロフェッショナル	https://www.soumu.go.jp/main_content/000447090.pdf
IBM	データに隠されている実用的な洞察を、専門知識、数学と統計、特殊プログラミング、高度な分析、人工知能（AI）、機械学習を組み合わせて明らかにする人材	https://www.ibm.com/jp-ja/topics/data-science
Microsoft	ビッグデータから有益な情報を抽出する研究プロジェクトを主導しており、テクノロジ、数学、ビジネス、およびコミュニケーションのスキルを有しています。組織はこの情報を使用して、より良い意思決定を行い、複雑な問題を解決し、業務運営を向上させます。	https://azure.microsoft.com/ja-jp/resources/cloud-computing-dictionary/what-is-data-science
株式会社ブレインパッド	データから引き出す価値を最大化するために、統計学や数学、プログラミングなどの技術を駆使してデータ活用を実践するスペシャリスト	https://www.brainpad.co.jp/doors/contents/01_about_data_scientist/

講師としては・・・

川口はIBMの定義がいちばんしっくりきます。所属組織だからかもしれませんが、前職の経験と合わせても違和感ないです。

組織	説明文	ソース
情報処理推進機構（IPA）	DXの推進において、データを活用した業務変革や新規ビジネスの実現に向けて、データを収集・解析する仕組みの設計・実装・運用を担う人材	https://www.ipa.go.jp/jinzai/skill-standard/dss/datascientist/index.html
一般社団法人データサイエンティスト協会	データサイエンス力、データエンジニアリング力をベースにデータから価値を創出し、ビジネス課題に答えを出すプロフェッショナル	https://www.soumu.go.jp/main_content/000447090.pdf
IBM	データに隠されている実用的な洞察を、専門知識、数学と統計、特殊プログラミング、高度な分析、人工知能（AI）、機械学習を組み合わせて明らかにする人材	https://www.ibm.com/jp-ja/topics/data-science
Microsoft	ビッグデータから有益な情報を抽出する研究プロジェクトを主導しており、テクノロジ、数学、ビジネス、およびコミュニケーションのスキルを有しています。組織はこの情報を使用して、より良い意思決定を行い、複雑な問題を解決し、業務運営を向上させます。	https://azure.microsoft.com/ja-jp/resources/cloud-computing-dictionary/what-is-data-science
株式会社ブレインパッド	データから引き出す価値を最大化するために、統計学や数学、プログラミングなどの技術を駆使してデータ活用を実践するスペシャリスト	https://www.brainpad.co.jp/doors/contents/01_about_data_scientist/

この講義でのデータサイエンティストの定義

川口担当分の講義では、データサイエンティストとは以下のものであるとします。

データに隠されている実用的な洞察を、専門知識、数学と統計、特殊プログラミング、高度な分析、人工知能（AI）、機械学習を組み合わせて明らかにする人材

※ これが正解ではありません。皆さんもデータサイエンティストになることがあれば、データサイエンティストは何なのかをしっかりと考えてみましょう。

データサイエンティストの活躍するフィールド

産業界のデータサイエンティストは、さまざまなフィールド・分野で活躍しています。

- 情報通信技術の発展により、分野によらず様々な大量のデータを保持するように
- そしてそのデータを競争力のために利活用する動きが活発化

色々なデータサイエンティスト

データサイエンティストの職務内容は、（当たり前だが）関わる業界によって違う

例えば、金融業界にいるのであれば、金融に関する分析が多くなる。
ちなみに、金融分析に特化したデータサイエンティストをクオンツと言ったりします。

この講義でのデータサイエンティスト

業界によって職務や求められるスキルが多少異なるのですが、ある程度方向性を決めないと学びづらいです。

この講義では、**ITベンダーで活躍するデータサイエンティスト**を目指します。

対象はITベンダーに絞りますが、とはいえある程度どの業界でも役立つであろうスキルを習得していきます。

※ ITベンダー: IT関連の製品やサービスを提供する企業

本講義で学ぶデータサイエンティストのまとめ

以上のことをまとめると、本講義で学ぶデータサイエンティストは、

IT関連の製品やサービスを提供するITベンダーで活躍する、データに隠されている実用的な洞察を、専門知識、数学と統計、特殊プログラミング、高度な分析、人工知能（AI）、機械学習を組み合わせて明らかにする人材

長いので短くすると・・・

ITベンダーで活躍する、データに隠されている実用的な洞察をいろいろな手段で明らかにする人材

データサイエンティストをより理解するために

データサイエンティストについてより理解するために、以下のことも学んでいきましょう

- ITベンダーとは
- データサイエンティスト以外のロール（役割・職種）について

ITベンダーにおけるビジネスや他のロールとの違いにフォーカスを当てた解説をしていきます。

ITベンダーのビジネス

ITベンダーは企業なので、利益を得るために売上を得る必要があります。そのためには、いろいろな手段がありますが、ここではざっくりと以下の２つに分けます。

- クライアントワーク
  お客さんに依頼された仕事を実行することでお金を得る仕事全般のことを指します。例えば、お客様に相談された問題について、情報技術を活かして解決すること等が該当します。

- 自社製品開発
  自社で何らかのソフトウェアやハードウェアを開発して、それを他社に販売すること

クライアントワークと自社製品開発の違い

- クライアントワークはフロー収入
  「その都度」収益を得る収入形態

- 自社製品開発はストック収入
  「継続的に」収益を得る収入形態

クライアントワークにおけるデータサイエンティスト

データサイエンティストでは、**クライアントワークと自社製品開発のどちらでも活躍できますが、おそらく多数派のクライアントワークに絞って解説していきます。**

ITベンダーにおけるクライアントワーク

ITベンダーにおけるクライアントワークは、お客さんに依頼された問題解決を情報技術を活かして解決する。

基本的には、何らかの問題を解決する情報システムを開発する。

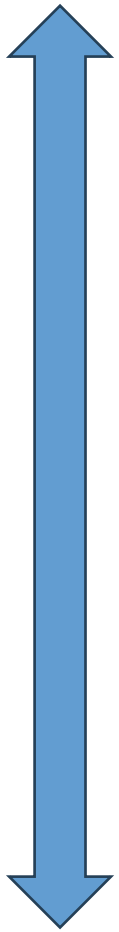
最近は、**データを利活用するために、AI・機械学習・分析 が組み込まれるシステムが開発されることが増えてきました。**

⇨ **データサイエンティストの出番が増えてきた**

情報システム開発におけるロール

ITベンダーでは、色々なロールの人々がチームを組んで仕事をします。

- 営業
- プロジェクトマネージャー
- ITコンサルタント
- データサイエンティスト
- ITエンジニア
  - アプリケーションエンジニア
  - AIエンジニア
  - 機械学習エンジニア
  - データエンジニア
  - インフラエンジニア
- 研究者（リサーチャー）



お客さんとよりコミュニケーションをとる

裏方として技術を活かした仕事をする

営業

最もお客さんに近い立場で活動するロールであり、ITベンダーにとっての「入り口」の役割を果たす。

- お客さんの課題やニーズのヒアリング
- 自社サービス・製品の提案
- 技術的な内容についてもある程度理解していることが望ましい
- 社内のエンジニアやコンサルタントと連携する

プロジェクトマネージャー

プロジェクトマネージャー(PM)は、提案が成約した後、実際にプロジェクトを進行する際の全体的な管理を担当します。

- 予算・スケジュール・品質・リスクといった複数の観点から、チーム全体を調整
- 関係者の利害を調整するコミュニケーション能力が必要
- トラブル時に適切な打ち手を講じられるマネジメント能力も必要
- データサイエンティストもプロジェクトの一員としてPMの管理下に入る

ITコンサルタント

ITコンサルタントは、お客さんの経営課題や業務プロセスを深く理解し、それを解決するためにどのようなIT活用が最も効果的かを検討して提案する役割を担う

- 「なぜそれが必要か」「どうやって導入すべきか」といった根本的な問いに答える
- 業務知識と情報技術の両方に精通している必要あり
- ビジネスとテクノロジーを橋渡しする存在
- データサイエンティストが設計・開発するソリューションは、コンサルタントの提案をもとに方向性が定まることも多い

データサイエンティスト

データに隠されている実用的な洞察をいろいろな手段で明らかにする人材（再掲載）

- データを分析し、実用的な洞察を得ることで課題解決に貢献する
  - データの傾向を分析し理解する
  - AI・機械学習を適用した場合の効果を測定する
- 分析データは構造化データ・テキストや画像といった非構造化データも含まれる
- 結果を関係者にわかりやすく伝える能力も重要

ITエンジニア

ITエンジニアは、実際にシステムを作り上げる技術者の総称。

- 専門によって、さらに細かく細分化される
- ITベンダーの中核的役割
- データサイエンティストとの連携も密に行われる。

※ 後のスライドで細分化したロールの説明をします。

研究者（リサーチャー）

研究者は、最先端の理論や技術を探究し、それを中長期的に企業の競争力へと繋げる役割を担う

- ITベンダーも、AI・セキュリティ・量子計算・ソフトウェア工学等の専門家が在籍している（場合がある）
- 成果は特許・論文・プロトタイプ
- クライアントワークで現れることは正直なところ稀
  - 異なる時間で活動しているイメージ

兼務

以上のロールが満遍なくきっちりと役割を分担していることは珍しいです。実際には暗黙的に複数の役割を兼務していることが多いです。

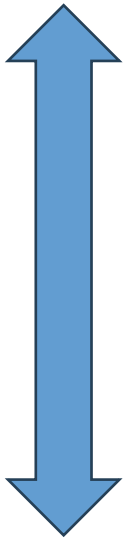
よくある兼務

- 営業 ⇨ ITコンサルタント
- プロジェクトマネージャー ⇨ ITコンサルタント
- ITコンサルタント ⇨ プロジェクトマネージャー
- データサイエンティスト ⇨ ITコンサルタント・ITエンジニア
- ITエンジニア ⇨ データサイエンティスト
- 研究者 ⇨ データサイエンティスト・ITエンジニア

色々なITエンジニア

ITベンダーには（当然ですが）ITエンジニアがたくさんいます。あとでデータサイエンティストとの違いを確認するために、ざっとそれぞれどんな役割を担うか解説していきます。

- アプリケーションエンジニア
- AIエンジニア
- 機械学習エンジニア
- データエンジニア
- インフラエンジニア



（他にも色々といます）

アプリケーションエンジニア

アプリケーションエンジニアは、ユーザーが直接操作する業務アプリケーションやWebシステムを設計・開発・運用する技術者です。

- 画面や処理の流れを設計し、プログラミング言語（Java, Python, JavaScriptなど）を用いて具体的な機能を実装する
- ユーザーインターフェースの使いやすさや、処理の正確性、セキュリティの担保など、多面的な視点が求められる
- 実際にユーザーの目に触れるシステムを形にする役割を担う

AIエンジニア

AIエンジニアは、画像認識・音声認識・自然言語処理など、人工知能（AI）技術を活用したアプリケーションを開発するエンジニア

- 機械学習エンジニアより応用面・製品レベルでのAI活用にフォーカス
  - 生成AIを用いた会話システムの開発
  - 監視カメラ映像から異常を検出するシステムの開発
- AI・機械学習の理論に加えて、ユーザ体験(UX)やリアルタイム性の考慮など、プロダクト設計に近い発想が必要とされる

機械学習エンジニア

機械学習エンジニアは、データサイエンティストが設計・構築したモデルを実際のシステムに組み込み、安定して稼働するように最適化する専門職

- 例えば以下の業務がある
  - ローカル環境でうまく動作していたモデルを、Webアプリケーションのバックエンドに API として組み込む作業
  - クラウド上でスケーラブルに動作させる環境構築
  - モデルの学習・推論・モデリングの一連の流れを継続的に維持する環境を整える（MLOps, 機械学習の運用）
- 機械学習の理論だけでなく、インフラやCI/CDの知識も必要とされる

データエンジニア

データエンジニアは、データを効率的・安全に収集・蓄積・加工・配信するための仕組みを整備するエンジニア

- 主な業務
  - ETL (Extract, Transform, Load)の設計
  - データレイクやデータウェアハウスの構築
  - パイプライン処理の自動化
- データサイエンティストが分析しやすいように、構造化されたクリーンなデータを用意するのが役割
- SQLやPython、クラウドサービスへの理解が必要

インフラエンジニア

インフラエンジニアは、システムを支えるネットワーク・サーバ・ストレージなどの基盤を構築・運用する職種

- 近年では、クラウドインフラ（AWS, Azure, GCPなど)を用いた設計・構築が主流になりつつある
- IaC (Infrastructure as Code)の考え方に基づいた自動化も進んでいる
- システムが高負荷でも落ちないようにする冗長構成や、セキュリティの確保、障害時の復旧設計などが担当領域

データサイエンティストとITエンジニアの違いとは？

データサイエンティストと ITエンジニア、特にAI/機械学習エンジニアの根本的な違いとは何でしょうか

- この講義では、データサイエンティストはITエンジニアとは違う区分にしています
- どちらも、AI・機械学習を専門とする
- どちらも、プログラミングはする

データサイエンティストとAI/機械学習エンジニアの違い

データサイエンティストとAI/機械学習エンジニアの最も大きな違いは、主な成果物です。

ロール	主な成果物	説明
データサイエンティスト	• ドキュメント	分析や機械学習モデルの作成を通じて得た知見を説明してプロジェクトの方向性等の「意思決定」を促す
AI/機械学習エンジニア	• システム • 機械学習モデル • コード	AIシステムや機械学習システムといった、実際にコンピュータ上で動く「モノ」を作る

成果物が違うことを意識することの大切さ

主な成果物が違うということは、**特に求められるスキルが違ってきます。**

- データサイエンティスト
  専門性に加えて、ドキュメンテーションスキルが特に必要
  - 論理的思考力
  - 文章執筆能力
  - プレゼンテーション能力

- AI/機械学習エンジニア
  専門性に加えて、ソフトウェアエンジニアリングスキルが特に必要
  - システム設計力
  - アルゴリズムの設計力
  - コーディング能力

その他に必要な能力

実際には兼務することも多いので、以下のことは注意しましょう。

- データサイエンティストに以下の能力が必要ないわけではない
  - システム設計力
  - アルゴリズムの設計力
  - コーディング能力
- AI/機械学習エンジニアに以下の能力が必要ないわけではない
  - 論理的思考力
  - 文章執筆能力
  - プレゼンテーション能力

データサイエンティストにもモノづくりのためのスキルはある程度必要だし、AI/機械学習エンジニアにも人に説明するスキルは必要です。

データサイエンティストとITコンサルタント/研究員

ITコンサルタントと研究員もドキュメントを書くことが大事なので、
データサイエンティストは実はこれらのロールと似ています。

- ITコンサルタントの成果物
  ドキュメントを通して以下を行う
  - お客さんへの提案
  - 検討結果の報告
- 研究者の成果物
  研究結果を以下の形としてまとめる
  - 学術論文
  - 特許原稿
  - 社内資料

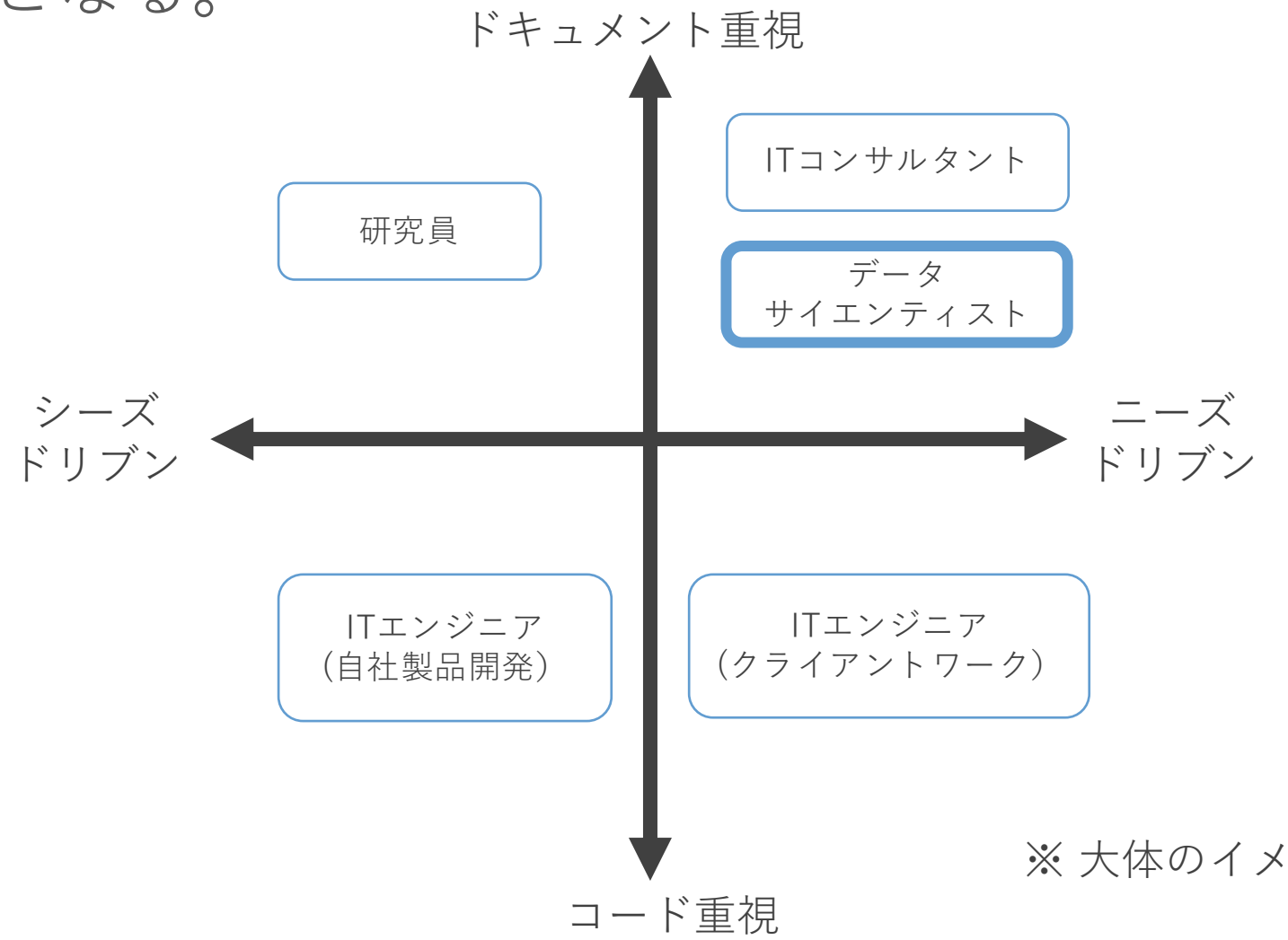
ITコンサルタント/研究員との違い

- データサイエンティストとITコンサルタントの違い
  - プログラミングをするかは明確に違うポイント
  - ただしデータサイエンティストはコードを書けるITコンサルタントとは言い難い
  - ITコンサルタントと比較して、戦略的思考やドキュメント作成能力が低いことが多い
  - またプログラミング能力も、ITエンジニアと比べて低いことが多い

- データサイエンティストと研究者の違い
  - データサイエンティストはお客さんの「ニーズ」に答える
  - 研究者は、会社の武器になる「シーズ」を作る

データサイエンティストと他のロールの関係

データサイエンティストと他のロールの関係を整理すると、以下のようなイメージとなる。



※ 大体のイメージです

あと３回の講義で学ぶこと

データサイエンティストに必要なドキュメントを書くスキル、ドキュメントに反映するための知見を得るための実践的な分析の管理方法、そして最低限身に付けたい開発技術について学びます。

- ドキュメント・コミュニケーション
  データサイエンティストが知っておくべきドキュメントの書き方やプレゼンテーション資料の作成方法について学ぶ

- 分析・実験・検証の実践的管理方法
  データサイエンティストの実施する仮説検証のプラクティカルな管理方法について学ぶ。

- データサイエンティストの開発技術
  データサイエンティストが開発現場で最低限知っておきたい実践的な技術について学ぶ。

今回の講義の目次

- [前半] イントロダクション
  - 本講義で学ぶデータサイエンティストとは何か
  - ITベンダーのデータサイエンティストの仕事
  - ITベンダーのデータサイエンティストと周囲の人々
- **[後半] 2回目以降の講義のためのセットアップ・前提知識の解説**
  - **Pythonのバージョン管理**
  - **仮想環境の管理**

２回目以降の講義のためのセットアップ・前提知識の解説のゴール

このパートでは、以下の２つをゴールとして、実践的に進めます。

- 実務で必須なPythonの実行環境管理の基礎知識の理解
  - Pythonインタプリタのバージョン管理
  - 仮想環境の管理
- 2回目以降（正確には３回目以降）の講義を実施できるように環境を整える

講義を進めるための環境・事前知識

講義を進めるためには、以下の環境が必要です。

- Python + 仮想環境(pip でパッケージをインストールできる)
- エディタ

あると良い知識

- Python の基本的な構文(If, for, 関数定義)
  - （とはいえ、簡単なのでC, Java等が分かれば楽勝）
- 機械学習、特に教師あり学習

データサイエンティストが主に使うプログラミング言語

データサイエンティストが最も多く使うプログラミング言語は、Pythonです。デファクトスタンダードです。

その他の言語

- SQL: データベース操作用（正確にはプログラミング言語ではない）
- R: 統計解析に強いが、産業界ではあまり使われるのを見たことがない

Pythonが人気の理由

なぜPythonがここまで人気なのかは講師も厳密なことは不明ですが、以下の理由はあるかと思います。

- 豊富なライブラリ・フレームワーク
  - Numpy, pandas -> データ処理・分析
  - Scikit-learn -> 機械学習
  - PyTorch -> 深層学習
  - Matplotlib, seaborn -> データ可視化
- 学習コストが低い
- 幅広い応用分野

とりあえずPythonが出来れば問題ありません。
この講義でも、Pythonを使っていきます。

Pythonの特徴

Pythonは、C言語やJavaとは異なる特徴を持っています。

1. インタプリタ型言語
   - コードを１行ずつ逐次実行する（コンパイル不要）
   - 対話的に動かせる
   - 試行錯誤がしやすい
   - インタプリタとは
     ソースコードを読みながら逐次解釈して実行するソフトウェア

2. 動的型付け言語
   - 変数に型を明示しなくてもよい。例えば、以下の型宣言がいらない

```
x = 10          # 整数
y = “text”      # 文字列
```

Pythonの実行環境

Pythonを使う上では、その実行環境の管理方法について基本的なことを知っておくことが重要です。

- **Pythonインタプリタのバージョン管理**
  複数バージョンのPythonをインストールしておき、適宜切り替え可能にする
- **仮想環境**
  ソフトウェア開発において、特定のプロジェクト専用に独立した実行環境を構築・管理する仕組み

注意

Pythonの実行環境の管理にはいろいろなツールがあり、流行がある。
→ 数年後には違う方法が普通になっているかも

この講義では、最新のツールの使い方を教えるのではなく、なるべく標準的で根本的な考え方から教えたいと思います。

なぜPythonのバージョン管理が必要か

実務でも、プロジェクトごとに異なるPythonバージョンを要求されます。

- プロジェクトAでは 3.11
- プロジェクトBでは 3.12

システム全体のPythonを変更すると他のプロジェクトで困る
→ そこで、Pythonのバージョンを内部で管理することが必要

仮想環境とは

仮想環境(Virtual Environment)とは、ソフトウェア開発において、特定のプロジェクト専用に独立した実行環境を構築・管理する仕組みのことを指します。

プロジェクトごとにライブラリのバージョンや設定を分離して管理することで、以下のメリットがあります。

- 他プロジェクトとの依存関係の競合を回避できる
- 開発環境の再現性を確保できる
- 環境の構築や共有、不要になったときの廃棄が容易になる

最も基本的なPythonの実行環境管理ツール

- Pythonインタプリタのバージョン管理
  pyenv
  - 長年使われているシンプルなPythonインタプリタのバージョン管理ツール
  - 複数のPythonインタプリタのバージョンをインストールしておくことができ、切り替え機能を可能にする
  - バージョン管理だけするツールはpyenvだけ
    （conda や uv といったツールでもできるが、そのほかにもいろいろな機能がある）

- 仮想環境
  venv
  Python公式が提供している仮想環境管理ツール。プロジェクトごとに違うパッケージ・ライブラリをインストール可能にする

実践パート

目的: pyenv, venvをインストールしてPythonの実行環境を整える

1. pyenvをインストール
2. Pythonをインストールして管理する
3. venvで仮想環境を作る
4. サンプルコードを動かす

1. pyenv をインストール

pyenvをインストールしましょう。使用OSによって多少インストール方法が異なります。

この資料では以下の2つの方法について書いています。

- Macの場合
- Ubuntuの場合
- Windowsの場合（実質 Ubuntu）

以下のコマンドでバージョン情報を確認することで、インストールの完了確認ができます。

```
pyenv  --version
```

1. pyenv をインストール (Macの場合)

以下のコマンドを実行してください。

1. Homebrewをインストール（未導入なら）

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Pythonのビルドに必要なパッケージとpyenvをインストール

```
brew install openssl readline sqlite3 xz zlib
```

```
brew install pyenv
```

3. シェル設定ファイル(~/.zshrc または ~/.bashrc)に設定を追記

```
export PYENV_ROOT="$HOME/.pyenv"
export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

4. 反映

```
source ~/.zshrc  # または source ~/.bashrc
```

1. pyenv をインストール (Ubuntuの場合)

以下のコマンドを実行してください。

1. 必要な依存パッケージをインストール

```
sudo apt update
```

```
sudo apt install -y make build-essential libssl-dev zlib1g-dev libbz2-dev libreadline-dev libsqlite3-dev wget
curl llvm libncursesw5-dev xz-utils  tk-dev libxml2-dev libxmlsec1-dev libffi-dev  liblzma-dev git
```

2. pyenv をクローンしてインストール

```
git clone https://github.com/pyenv/pyenv.git ~/.pyenv
```

3. シェル設定ファイル(~/.bashrc または ~/.zshrc)に設定を追記

```
export PYENV_ROOT="$HOME/.pyenv"
export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
```

4. 反映

```
source ~/.bashrc   # または source ~/.zshrc
```

Windowsの場合

Windowsの場合は、WSL2上でUbuntuを動かした方が良いので、まずはUbuntuをインストールして、その後に１つ前のスライドの方法でインストールしましょう。

1. スタートメニューバーで「Windowsの機能の有効化または無効化」と検索して、開く
2. 「Linux用 Windowsサブシステム」にチェックを入れ、「OK」を押す
3. 再起動を促される場合は再起動する
4. スタートメニューバーで「 Ubuntu 24.04.1 LTS」と検索する
5. 「ストア」というカテゴリに「Ubuntu 24.04.1 LTS」が現れるので、そこを押下して画面の指示に従いインストールする
   - （もしくはMicrosoft Store からインストールする）

pyenvのコマンド一覧

- インストールできるバージョン一覧の表示

```
pyenv install --list
```

- Pythonのインストール

```
pyenv install <バージョン名>    # バージョン名は、pyenv install –list で表示された名前のいずれかを入れる
```

- インストールしたPythonのバージョン一覧を確認

```
pyenv versions
```

- システム全体で使うバージョンを設定

```
pyenv global <バージョン名>
```

- カレントディレクトリ限定のバージョンを設定
（.python-version が作られる）

```
pyenv local <バージョン名>
```

2. Pythonをインストールして管理する

以下を行い、条件によって使用されるPythonのバージョンが異なることを確認します。

- 2種類のバージョンをインストール
  - 3.11.13
  - 3.12.11
- システム全体と特定のフォルダ内の利用設定を行う

[実践] pyenvによるバージョン管理の実践 (1)

1. `pyenv install --list`で、以下のバージョンがあることを確認しましょう。
   - 3.11.13
   - 3.12.11
2. 以下の2つのコマンドで 2種類のバージョンをインストールしましょう。（数分はかかるかも）
   - `pyenv install 3.11.13`
   - `pyenv install 3.12.11`

ない場合は、別のPythonのバージョンでも大丈夫です。
その場合は、以降の資料はそのバージョンに読み替えてください。

3. `pyenv versions`でインストールされたバージョン一覧を確認
   以下のようになっていればOK

```
hidetoshi@Hidetoshi:~$ pyenv versions
* system (set by /home/hidetoshi/.pyenv/version)
  3.11.13
  3.12.11
```

[実践] pyenvによるバージョン管理の実践 (2)

4. `pyenv global 3.12.11`でシステム全体で使われるバージョンを3.12.11に変更
5. `pyenv versions` で、今利用されるPythonのバージョンが変わっていることを確かめる（アスタリスクマークがつきます）

```
hidetoshi@Hidetoshi:~$ pyenv versions
  system
  3.11.13
* 3.12.11 (set by /home/hidetoshi/.pyenv/version)
```

6. `python --version`で、実行するpythonのバージョンを確かめる

```
hidetoshi@Hidetoshi:~$ python --version
Python 3.12.11
```

[実践] pyenvによるバージョン管理の実践 (3)

7. `mkdir practical-ds-exercises`で講義用フォルダを作成
   - 作成場所・フォルダ名は任意です
   - 資料上は「 practical-ds-exercises」とします。
8. `cd practical-ds-exercises` で講義用フォルダに移動
9. `pyenv local 3.11.13`を実行して、講義用フォルダ内で実行される python のバージョンを 3.11.13に変更する
10. `python --version` を実行して、Pythonのバージョンが3.11.13になっていることを確認する。

```
hidetoshi@Hidetoshi:~/practical-ds-exercises$ python --version
Python 3.11.13
```

[実践] pyenvによるバージョン管理の実践 (4)

11. 講義用フォルダの１つ上に移動して、再度 pythonのバージョンを確認してみましょう。

```
hidetoshi@Hidetoshi:~$ python --version
Python 3.12.11
```

ローカルのPythonバージョンを設定したところと、していないところでPythonのバージョンが違っていますね。これが重要

pyenv の globalとlocalの切り替えの仕組み (1)

pyenv で管理されているpythonを実行するたびに、以下の優先順位でバージョンを確認し、決定しています。

1. カレントディレクトリの **.python-version** というファイルに書かれているバージョン名を使用

```
hidetoshi@Hidetoshi:~/practical-ds-exercises$ cat .python-version
3.11.13
```

2. 1.で.python-versionが見つからないなら、親ディレクトリの .python-verionを確認。以降見つかるまでこの手順を繰り返す
3. 2.で.python-versionが見つからなかった場合は、 **$PYENV_ROOT にある (一番多いのはホームディレクトリの.pyenvフォルダ) version** ファイルに書かれているバージョン名を使用

```
hidetoshi@Hidetoshi:~$ cat "$PYENV_ROOT/version"
3.12.11
```

pyenv の globalとlocalの切り替えの仕組み (2)

以上の仕組みを踏まえると、以下のことが分かる

- `pyenv local <バージョン名>` コマンドはカレントディレクトリにバージョン名が書かれた .python-version という名前のテキストファイルを作成する
- `pyenv global <バージョン名>`コマンドは、$PYENV_ROOTにある version ファイルの内容を、指定したバージョン名に書き換える（または書いた状態で作成する）

3. venvで仮想環境を作る

venv は、Python公式が提供する仮想環境作成機能です。

- 多くの仮想環境・パッケージ管理ツールではこの venv をベースに色々な機能を付加している（venvのラッパー）
- venv を知っていれば、何かと応用が効く
- 標準機能なので、Pythonと同時にインストールされる（追加インストールの必要はなし）

[実践]仮想環境・パッケージ管理 (1)

1. 先の実践で作成した講義用ディレクトリ(この資料上はpractical-ds-exercises)に移動
   - `cd practical-ds-exercises`
2. project-a と project-b という二つのディレクトリを作成
   - `mkdir project-a`
   - `mkdir project-b`
3. project-a ディレクトリに移動
   - `cd project-a`
4. **`python –m venv .venv-a` コマンドを実行して仮想環境を作る**
   - これは現在のPythonを基に仮想環境を作る
     この講義では、親ディレクトリのpractical-ds-exercises に設定されているPython 3.11.13が基になるはず
   - .venv-a フォルダがプロジェクト直下に作られます。
     この.venv-aフォルダ内にパッケージがインストールされたり設定が管理される

[実践]仮想環境・パッケージ管理 (2)

5. `source .venv-a/bin/activate` コマンドを実行して、仮想環境を有効化する
   - プロンプトの頭に (.venv-a) がつくことを確認する。これが今どの仮想環境が有効化されているかを示している。

```
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ source .venv-a/bin/activate
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$
```

6. 以下のコマンドを実行して、状態を確かめる。
   - `python –version`
   - `which python`

pyenvで設定しているバージョン

```
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ python --version
Python 3.11.13
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ which python
/home/hidetoshi/practical-ds-exercises/project-a/.venv-a/bin/python
```

Pythonインタプリタ自体は、.venv-aの中を指している
（実体はpyenvで管理しているpythonへのシンボリックリンク

pipとは？

pipとは、Pythonの公式パッケージ管理ツールです。

- ライブラリ（外部パッケージ）に対して以下のことができる
  - インストール: `pip install <パッケージ名>`
  - アップデート: `pip install –U <パッケージ名>`
  - アンインストール: `pip uninstall <パッケージ名>`
  - インストール済み一覧表示: `pip freeze`
- Pythonに標準で付属している
- 仮想環境ごとに独立したpip環境がある

[実践]仮想環境・パッケージ管理 (3)

7. `pip install numpy` コマンドを実行して、仮想環境(.venv-a)にnumpyというライブラリをインストールする。

```
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ pip install numpy
Collecting numpy
  Downloading numpy-2.3.3-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.1/62.1 kB 626.8 kB/s eta 0:00:00
Downloading numpy-2.3.3-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.9/16.9 MB 18.1 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.3.3
```

8. `python` コマンドを入力して、対話形式のインタプリタを開く

```
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ python
Python 3.11.13 (main, Oct  5 2025, 16:47:54) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

[実践]仮想環境・パッケージ管理 (4)

9. 以下を入力して、Numpyが使えることをたしかめてPythonインタプリタを終了する
   - `import numpy as np`
   - `np.array([1, 2, 3])`
   - `exit(0)`

```
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ python
Python 3.11.13 (main, Oct  5 2025, 16:47:54) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np
>>> np.array([1, 2, 3])
array([1, 2, 3])
>>> exit(0)
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$
```

10. `deactivate`を入力して仮想環境を終了する

```
(.venv-a) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$ deactivate
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-a$
```

[実践]仮想環境・パッケージ管理 (5)

11. `cd ../project-b` で移動する
12. `pyenv install 3.13.7`でPythonのバージョン3.13.7をインストールする。（project-bでは3.13.7を利用するという想定）

```
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ pyenv install 3.13.7
Downloading Python-3.13.7.tar.xz...
-> https://www.python.org/ftp/python/3.13.7/Python-3.13.7.tar.xz
Installing Python-3.13.7...
Installed Python-3.13.7 to /home/hidetoshi/.pyenv/versions/3.13.7
```

13. `pyenv local 3.13.7` でproject-bのPythonバージョンを設定

```
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ pyenv local 3.13.7
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ python --version
Python 3.13.7
```

[実践]仮想環境・パッケージ管理 (6)

14. 以下のコマンドで project-b用に仮想環境を作成し有効化する。

```
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ python -m venv .venv-b
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ source .venv-b/bin/activate
(.venv-b) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$
```

15. `pip install requests`でrequestsというライブラリをインストール

```
(.venv-b) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ pip install requests
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'Pr
otocolError('Connection aborted.', ConnectionResetError(104, 'Connection reset by peer'))': /packages/f5/83/6ab5883f57c9
c801ce5e5677242328aa45592be8a00644310a008d04f922/charset_normalizer-3.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_1
7_x86_64.manylinux_2_28_x86_64.whl.metadata
  Downloading charset_normalizer-3.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.
metadata (37 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Using cached urllib3-2.5.0-py3-none-any.whl.metadata (6.5 kB)
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2025.10.5-py3-none-any.whl.metadata (2.5 kB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.4-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (1
53 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Using cached urllib3-2.5.0-py3-none-any.whl (129 kB)
Using cached certifi-2025.10.5-py3-none-any.whl (163 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, requests
Successfully installed certifi-2025.10.5 charset_normalizer-3.4.4 idna-3.11 requests-2.32.5 urllib3-2.5.0
```

Warning が出ても今は気にしないでください。
（実務では気にしましょう）

[実践]仮想環境・パッケージ管理 (7)

16. 以下のコマンドで 対話形式pythonインタプリタでrequestsを使ってパッケージがインストールされて使えることを確かめましょう。
    - `python`
    - `import requests`
    - `requests.get("https://example.com")

```
(.venv-b) hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ python
Python 3.13.7 (main, Oct  8 2025, 22:19:40) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import requests
>>> requests.get("https://example.com")
<Response [200]>
```

[実践]仮想環境・パッケージ管理 (8)

17. `import numpy as np`を実行して、project-a (.venv-a)でインストールしていた Numpyが使えないことを確かめましょう。

```
>>> import numpy as np
Traceback (most recent call last):
  File "<python-input-2>", line 1, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
>>>
```

18. 以下のコマンドを実行して、インタプリタを終了し、仮想環境(.venv-b)を終了しましょう。
    - `exit(0)`
    - `deactivate`

venv 仮想環境の削除方法

venv で作成した仮想環境は、**`python –m venv <仮想環境名>` で作成されたフォルダを `rm –r <仮想環境名>` として削除できます。**

例えば、さっき作ったproject-b の場合は、以下のようにすれば削除できます。

```
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ ls -la
total 16
drwxr-xr-x 3 hidetoshi hidetoshi 4096 Oct  8 22:49 .
drwxr-xr-x 4 hidetoshi hidetoshi 4096 Oct  5 18:41 ..
-rw-r--r-- 1 hidetoshi hidetoshi    7 Oct  8 22:20 .python-version
drwxr-xr-x 5 hidetoshi hidetoshi 4096 Oct  8 22:49 .venv-b
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ rm -r .venv-b/
hidetoshi@Hidetoshi:~/practical-ds-exercises/project-b$ ls -la
total 12
drwxr-xr-x 2 hidetoshi hidetoshi 4096 Oct  8 22:49 .
drwxr-xr-x 4 hidetoshi hidetoshi 4096 Oct  5 18:41 ..
-rw-r--r-- 1 hidetoshi hidetoshi    7 Oct  8 22:20 .python-version
```

流行の仮想環境の変遷

Pythonの仮想環境には流行り廃りがある

- 2020年まで
  Anacondaというデータサイエンス向けの統合ディストリビューションが標準的に使われていた（少なくとも研究・データサイエンス界隈では）

- 2021年
  Anaconda のライセンス変更で大き目の企業は使用を制限され下火に
  - 「従業員200名以上または売上1,000万ドル以上の企業」は有償化対象
  - 個人・教育・研究での使用は引き続き無料

- 2022年ごろから最近まで
  Minicondaという軽量なAnaconda(こちらはフリーで使える） や**Poetry**という venv のラッパーがよく使われるように

- 最近
  uv というPoetryと似た仮想環境ツールが流行中

流行り廃りはあるが・・・

pyenv と venv は常に選択肢のひとつになりますし、最近よく使われるPoetry や uv といった仮想環境管理ツールの下地として使われているので、使い方を知っておくと何かと役にたつ。

例えばPoetry は venv のラッパーなので、venv の知識が役立つ

最新のツールを使うのはもちろん結構だが、pyenv + venvの使い方は理解しておきましょう！

次回以降の講義でも、講義用の仮想環境を作って是非管理してみてください。

まとめ

- [前半] イントロダクション
  - 本講義で学ぶデータサイエンティストとは何か
    - ITベンダーで活躍する、データに隠されている実用的な洞察をいろいろな手段で明らかにする人材
  - ITベンダーのデータサイエンティストと周囲の人々
    - 営業・ITコンサルタント・ITエンジニア・研究者がいる
    - データサイエンティストは、コードも書くがドキュメントを成果として重視し、顧客のニーズドリブンで働く

- [後半] 2回目以降の講義のためのセットアップ・前提知識
  - Pythonのバージョン管理
    - pyenv の使い方は覚えておこう
  - 仮想環境の管理
    - Venv の使い方は覚えておこう

次回以降の予告

- **ドキュメント・コミュニケーション （2回目）**
  データサイエンティストが知っておくべきドキュメントの書き方やプレゼンテーション資料の作成方法について学ぶ

- **分析・実験・検証の実践的管理方法（3回目）**
  データサイエンティストの実施する仮説検証のプラクティカルな管理方法について学ぶ。

- **データサイエンティストの開発技術（４回目）**
  データサイエンティストが開発現場で最低限知っておきたい実践的な技術について学ぶ。

宿題

以下の２つのテーマについてそれぞれ400字以上でレポートにまとめて提出してください。

- 提出方法: 学内システムのレポート機能
- 締め切り: XXX

1. 今日の講義内容を踏まえて、将来どのようなキャリアを築きたいか説明してください。
   - データサイエンティストでも良いし、ITコンサルタントやITエンジニアでもよい
   - 今日紹介したロール以外でも良いが、その場合はデータサイエンティストやエンジニア等のよう紹介したロールとの比較に関する考察を述べること

2. Pythonのバージョン管理や仮想環境がなぜ重要なのか、講義内容を踏まえて説明してください。

Copyright © 2025 Hidetoshi Kawaguchi